COMPTIA SECURITY+ SY0-701 | DOMÍNIO 2 · AMEAÇAS, VULNERABILIDADES E MITIGAÇÕES (22%)

Security+ 2.3 · Race Conditions

Mestre CyberSec · **Aula 2.3 — Race Conditions** (4:58) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: Race condition: dois processos tentam MUDAR O MESMO recurso ao mesmo tempo — o resultado depende da ordem. TOCTOU (Time-of-Check to Time-of-Use): checa permissão em t1, executa em t2 — gap exploitação. Defesa: operação ATÔMICA, lock.

💡 Highlights que a prova CompTIA cobra

- Race condition: resultado depende da ordem de execução
- TOCTOU: gap entre checar permissão e usar o recurso

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'checks permission at time A but executes at time B' = TOCTOU

🚨 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 2.3 — Race Conditions)

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Condição de Corrida (N/A)

Vulnerabilidade em que múltiplos processos ou threads acessam e modificam o mesmo recurso ao mesmo tempo, fazendo o resultado de segurança depender da ordem imprevisível de execução.

⚠ ***Armadilha CompTIA:*** *CompTIA costuma descrever 'result depends on execution order' ou 'same resource changed at the same time'; não confunda com falta de atualização ou erro de permissão isolado.*

• Tempo de Verificação até o Tempo de Uso (TOCTOU)

Condição de corrida em que o sistema verifica uma condição ou permissão em um momento, mas o recurso muda antes de ser usado, permitindo ao atacante explorar essa janela.

⚠ ***Armadilha CompTIA:*** *Gatilho clássico: 'checks permission at time A but executes at time B' = TOCTOU. Palavra-chave: gap entre check e use.*

• Operação Atômica (N/A)

Operação que executa por completo ou não executa, sem estado intermediário visível a outros processos, impedindo interferência por condição de corrida.

⚠ ***Armadilha CompTIA:*** *Defesa-chave em race conditions: se a questão pede impedir mudança entre check e use, pense em operação atômica ou lock, não em criptografia.*

• Bloqueio / Lock (N/A)

Mecanismo de sincronização que permite apenas um processo ou thread acessar um recurso compartilhado por vez, evitando modificação concorrente.

⚠ ***Armadilha CompTIA:*** *Lock é mitigação direta de race condition; CompTIA pode citar mutex ou lock para garantir acesso exclusivo a recurso compartilhado.*

• Exclusão Mútua (Mutex) (MUTEX)

Mecanismo de bloqueio que garante que apenas uma thread entre em uma seção crítica ou acesse um recurso compartilhado por vez.

⚠ ***Armadilha CompTIA:*** *Mutex = lock de exclusão mútua. Não confunda com semáforo, que pode permitir múltiplos acessos limitados, nem com criptografia.*

• Seção Crítica (N/A)

Trecho de código que acessa um recurso compartilhado e deve ser protegido para que apenas um processo ou thread o execute por vez.

⚠️ ***Armadilha CompTIA:*** *Se a questão menciona código que altera recurso compartilhado, a proteção da seção crítica com lock ou atomicidade evita race condition.*

• Concorrência (N/A)

Execução de múltiplos processos ou threads que podem se sobrepor no tempo e competir por recursos compartilhados, criando risco de condição de corrida.

⚠️ ***Armadilha CompTIA:*** *Concorrência é o cenário que possibilita race condition; por si só não é a vulnerabilidade, mas a falta de sincronização é.*

• Operação Não Atômica (N/A)

Operação de múltiplas etapas que pode ser interrompida entre etapas, permitindo que outro processo altere o recurso e cause condição de corrida ou falha TOCTOU.

⚠️ ***Armadilha CompTIA:*** *TOCTOU geralmente envolve uma sequência não atômica de checar e depois usar; a correção é tornar a operação atômica ou protegida por lock.*

🏛️ Visão Didática, Analogias & Análise de Governança (GRC)

📌 TOCTOU (Time-of-Check to Time-of-Use)

🏠 **Analogia Intuitiva:** Você vai sacar 100 reais no caixa eletrônico. O sistema checa seu saldo às 12:00:00 (você tem 100 reais). Entre a checagem e a liberação das notas (12:00:01), você faz um saque simultâneo no celular. Se o banco não trancar a conta durante o segundo do saque, você consegue tirar 200 reais tendo apenas 100.

🔍 **Causa Raiz / Por que foi criado:** Múltiplos processos ou threads acessam o mesmo recurso compartilhado sem travas de sincronização (Locks / Mutexes).

🛡️ **Visão do Auditor de GRC:** Exigir revisões de código de concorrência e uso de transações atômicas com bloqueio estrito em operações financeiras e de privilégio.

⚠️ **Como cai na Prova CompTIA:** Trigger: 'vulnerability occurring between checking a permission and performing the action' -> Race Condition / TOCTOU.

⚡ TABELA DE FIXAÇÃO RÁPIDA PARA O EXAME (CHEAT SHEET)

Cenário Típico de Exame / Situação Prática	Conceito Técnico & Resposta da Banca
Dois eventos concorrentes competem pelo mesmo recurso criando estado inconsistente	**Race Condition (Condição de Corrida)**
Atacante altera um arquivo no intervalo entre a verificação de permissão e a leitura	**TOCTOU (Time-of-Check to Time-of-Use)**

✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Um desenvolvedor de uma fintech escreve um script que verifica o saldo da conta de um usuário antes de processar um saque. A consulta e o saque são duas chamadas separadas ao banco de dados. Um atacante descobre que pode enviar duas requisições simultâneas de saque pelo saldo total, e ambas transações são aprovadas porque a consulta do saldo acontece antes que qualquer saque seja registrado. Qual vulnerabilidade DESCREVE MELHOR esse problema?

A) Condição de corrida Time-of-Check to Time-of-Use (TOCTOU)

B) Injeção de SQL por entrada não sanitizada

C) Cross-site request forgery explorando cookies de sessão

D) Validação insuficiente de entrada no valor do saque

QUESTÃO 2 · CENÁRIO DE EXAME

Uma aplicação web permite que usuários resgatem um código de desconto de uso único. Um atacante usa um script para disparar 50 requisições HTTP no mesmo instante em que a promoção vai ao ar. Embora o código seja válido apenas uma vez, o atacante consegue aplicar o desconto em cinco pedidos separados. A causa raiz é que a verificação 'código já foi usado?' e a escrita 'marcar código como usado' acontecem como duas consultas independentes. Qual mitigação o desenvolvedor deve implementar?

A) Combinar a validação e o resgate em uma única transação atômica de banco de dados com lock em nível de linha

B) Adicionar CAPTCHA ao formulário de checkout para desacelerar requisições automatizadas

C) Aplicar hash no código de desconto antes de armazená-lo no banco de dados

D) Impor HTTPS no endpoint de checkout para criptografar o payload

QUESTÃO 3 · CENÁRIO DE EXAME

Um servidor Linux executa um cron job privilegiado que faz o seguinte: verifica se o arquivo /tmp/payload.sh pertence ao root e, em caso afirmativo, o executa. Um atacante local substitui /tmp/payload.sh por um script malicioso na pequena janela entre a verificação de propriedade e a execução. Qual é esse ataque e qual é a MELHOR correção?

A) Ataque TOCTOU; corrigir tornando a verificação-e-execução uma única operação atômica usando um descritor de arquivo (open + fstat + fexecve) em vez de chamadas separadas stat() e execve()

B) Ataque de buffer overflow; corrigir habilitando ASLR e stack canaries

C) Ataque de symlink race; corrigir desabilitando a execução em /tmp com uma montagem somente leitura

D) Escalada de privilégio por sudo mal configurado; corrigir editando o arquivo sudoers

QUESTÃO 4 · CENÁRIO DE EXAME

Uma biblioteca de autenticação permite que usuários alterem sua senha. O fluxo é: (1) verificar se a senha antiga corresponde ao hash armazenado, (2) se corresponder, sobrescrever o hash armazenado com o novo. Um pentester demonstra que, se um atacante envia a requisição de troca de senha exatamente no instante em que uma requisição legítima da vítima está em andamento, o atacante consegue sequestrar a conta. Que tipo de falha de concorrência está sendo explorada?

A) Condição de corrida em uma operação não atômica de atualização de credenciais, explorável entre os passos de verificação da antiga e gravação da nova

B) Ataque man-in-the-middle devido à ausência de TLS no endpoint de troca de senha

C) Ataque de replay contra o cookie de sessão usado pela API de troca de senha

D) Algoritmo de hash de senha fraco suscetível a quebra offline

QUESTÃO 5 · CENÁRIO DE EXAME

Uma revisão de segurança em um daemon C++ multithreaded encontra o seguinte padrão no módulo de controle de acesso: 'if (user.role == admin) { sleep(2); runAdminCommand(); }'. Duas threads avaliando o mesmo objeto de usuário simultaneamente podem observar role == admin antes que qualquer thread alcance runAdminCommand(). Qual construção de código seguro deve ser aplicada pelo desenvolvedor em toda a seção crítica?

A) Adquirir um mutex (lock de exclusão mútua) em torno da seção crítica de verificar-e-executar para que apenas uma thread por vez avalie o privilégio e atue sobre ele

B) Aumentar o tamanho do pool de threads para atender mais usuários concorrentes

C) Desabilitar threading definindo uma flag de build single-threaded

D) Substituir a verificação de role por um regex no nome de usuário

QUESTÃO 6 · CENÁRIO DE EXAME

A API de uma plataforma de e-commerce verifica se um produto está em estoque lendo de um cache e depois decrementa a contagem de inventário no banco. Durante uma promoção relâmpago, clientes relatam que itens marcados como 'em estoque' são vendidos várias vezes, gerando overselling. A leitura no cache e o decremento no banco não são atômicos. Qual controle ELIMINA MELHOR essa condição de corrida de overselling?

A) Usar um decremento atômico no banco (ex.: UPDATE stock SET qty = qty - 1 WHERE product_id = ? AND qty > 0) dentro de uma transação com lock, para que leitura e escrita sejam inseparáveis

B) Servir páginas estáticas de marketing a partir de uma CDN para reduzir a carga do backend durante a venda

C) Implementar afinidade de sessão baseada em cookie HTTP para rotear usuários ao mesmo nó backend

D) Migrar de REST para GraphQL para otimizar payloads de consulta

QUESTÃO 7 · CENÁRIO DE EXAME

Um analista de segurança revisa o código-fonte de uma biblioteca de gerenciamento de sessão e encontra o padrão: 'if (session.isValid()) { session.regenerateId(); performAction(); }'. Um atacante cronometra suas chamadas para que duas requisições compartilhem o mesmo ID de sessão válido no momento da regeneração, permitindo que uma requisição atue sob a identidade da vítima. Qual termo CLASSIFICA MELHOR essa falha?

A) Condição de corrida Time-of-Check to Time-of-Use sobre a validade da sessão, permitindo fixação de sessão ou confusão de identidade

B) Referência insegura direta a objeto (IDOR) por identificadores de sessão previsíveis

C) Cross-site scripting via dados de sessão não sanitizados em páginas de erro

D) Tratamento inadequado de erros vazando stack traces ao navegador

QUESTÃO 8 · CENÁRIO DE EXAME

Uma API REST implementa rate limiting lendo um contador de um arquivo, verificando se excede 100 e, caso não exceda, incrementando o contador e processando a requisição. Um atacante envia 200 requisições em paralelo e todas as 200 são processadas mesmo com o limite sendo 100. A correção proposta pela equipe é ler e incrementar o contador em um passo indivisível. A que conceito de segurança essa correção corresponde?

A) Operação atômica: combinar a leitura e o incremento em um único passo indivisível para que requisições concorrentes não possam se intercalar entre verificar e modificar

B) Criptografia simétrica do arquivo de contador em repouso usando AES-256

C) Controle de acesso baseado em papéis (RBAC) no endpoint da API aplicando somente leitura para guests

D) Infraestrutura de chave pública com mTLS X.509 entre cliente e servidor

QUESTÃO 9 · CENÁRIO DE EXAME

Uma função de nuvem provisiona automaticamente uma máquina virtual fazendo: (1) chama a API IAM para verificar se o chamador tem permissão 'compute.create', (2) chama a API de nuvem para realmente criar a VM. Um tester mostra que revogar a permissão do chamador entre os passos 1 e 2 ainda resulta em uma VM sendo criada, porque a decisão em cache do passo 1 é reutilizada. Qual controle ABORDA MAIS DIRETAMENTE essa falha de Time-of-Check to Time-of-Use?

A) Reverificar a autorização no momento da criação do recurso e exigir que a decisão de acesso esteja vinculada a um token assinado de uso único e curta duração obtido dentro da mesma transação

B) Rotacionar as chaves de API raiz do provedor de nuvem a cada 24 horas

C) Habilitar VPC flow logs no projeto para capturar todas as chamadas de API para revisão forense

D) Armazenar políticas IAM em um bucket do Cloud Storage público para transparência

QUESTÃO 10 · CENÁRIO DE EXAME

Um desenvolvedor de kernel está corrigindo um driver vulnerável que chama copy_from_user() para ler um buffer em userspace e depois usa o conteúdo do buffer em uma operação privilegiada. Um advisory de segurança observa que o argumento de tamanho passado a copy_from_user() é lido de uma variável em userspace que outra thread pode modificar concorrentemente. Qual mitigação elimina a corrida entre a validação do tamanho e o uso?

A) Copiar o tamanho para uma variável local do kernel, validá-lo uma vez e passar essa cópia local validada a copy_from_user(), para que o valor não mude entre a checagem e o uso

B) Desabilitar o escalonamento preemptivo na configuração do kernel para impedir trocas de thread

C) Criptografar a memória de userspace usando uma chave AES interna do kernel antes de copiar

D) Mover a lógica de cópia do buffer para userspace usando um wrapper de syscall

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa A — Condição de corrida Time-of-Check to Time-of-Use (TOCTOU)
A falha é uma condição de corrida TOCTOU clássica: o sistema verifica o saldo em t1 e usa (debita) em t2. Como são operações separadas e não atômicas, um atacante pode intercalar requisições para que ambas as verificações passem antes de qualquer débito. Injeção de SQL, CSRF e validação de entrada não estão relacionadas ao gap temporal entre verificação e uso. Operações atômicas (ou lock em nível de linha) devem envolver toda a sequência consultar-e-debitar para que as duas operações não possam ser separadas.

Questão 2: Alternativa A — Combinar a validação e o resgate em uma única transação atômica de banco de dados com lock em nível de linha
Transações atômicas com lock em nível de linha (ex.: SELECT ... FOR UPDATE) garantem que 'verificar usado?' e 'marcar usado?' não possam ser separadas entre requisições concorrentes, eliminando a janela TOCTOU. CAPTCHA apenas desacelera bots e não corrige a falha lógica. Hash trata confidencialidade, não condições de corrida. HTTPS protege dados em trânsito, mas não impede abuso lógico concorrente.

Questão 3: Alternativa A — Ataque TOCTOU; corrigir tornando a verificação-e-execução uma única operação atômica usando um descritor de arquivo (open + fstat + fexecve) em vez de chamadas separadas stat() e execve()
O script faz stat() em t1 e execve() em t2. Um atacante pode trocar o arquivo nesse intervalo — TOCTOU clássico. Usar open() para obter um descritor, depois fstat() e fexecve() sobre esse fd torna a operação atômica sobre o mesmo inode. Desabilitar /tmp atenua parcialmente, mas é específico de ambiente e frágil. Buffer overflow e sudoers são categorias não relacionadas.

Questão 4: Alternativa A — Condição de corrida em uma operação não atômica de atualização de credenciais, explorável entre os passos de verificação da antiga e gravação da nova
A 'verificação do hash antigo' e a 'escrita do hash novo' são passos não atômicos. Requisições concorrentes podem correr para que um atacante se encaixe na lacuna — um TOCTOU em estado compartilhado de credenciais. Mitigações incluem envolver ambos os passos em uma transação de banco com a linha bloqueada, ou gerar um nonce/ticket por requisição vinculado à credencial antiga verificada. MITM exige ausência de TLS, replay requer cookie roubado e hash fraco é quebra offline — nenhum trata gaps lógicos concorrentes.

Questão 5: Alternativa A — Adquirir um mutex (lock de exclusão mútua) em torno da seção crítica de verificar-e-executar para que apenas uma thread por vez avalie o privilégio e atue sobre ele
Um mutex em torno da seção crítica garante exclusão mútua: apenas uma thread mantém o lock, faz a verificação de role e executa o comando privilegiado de forma atômica — fechando a janela TOCTOU. Aumentar o pool piora a concorrência. Desabilitar threading é inviável em daemons de produção. Regex em username é questão de lógica/entrada, não de concorrência.

Questão 6: Alternativa A — Usar um decremento atômico no banco (ex.: UPDATE stock SET qty = qty - 1 WHERE product_id = ? AND qty > 0) dentro de uma transação com lock, para que leitura e escrita sejam inseparáveis
Um único UPDATE atômico com a condição WHERE qty > 0 realiza verificar-e-decrementar como uma operação indivisível. O motor do banco serializa updates conflitantes, então duas requisições concorrentes não podem passar ambas pelo 'em estoque?' e gerar overselling. Offload em CDN, sticky sessions e GraphQL tratam desempenho ou roteamento — não fecham o gap lógico TOCTOU entre leitura no cache e escrita no banco.

Questão 7: Alternativa A — Condição de corrida Time-of-Check to Time-of-Use sobre a validade da sessão, permitindo fixação de sessão ou confusão de identidade
A validade da sessão é verificada em t1 e a ação é executada em t2 usando um estado que pode mudar entre esses pontos — uma TOCTOU clássica. A regeneração atômica da sessão (ex.: vincular o novo ID ao nonce da requisição dentro de uma transação com lock) fecha a janela. IDOR trata referências de objetos, XSS trata codificação de saída e stack traces tratam tratamento de erros — nenhum corresponde à falha temporal verificar-para-usar.

Questão 8: Alternativa A — Operação atômica: combinar a leitura e o incremento em um único passo indivisível para que requisições concorrentes não possam se intercalar entre verificar e modificar
Operações atômicas (ou compare-and-swap / lock + read + write) eliminam a janela TOCTOU tornando toda a sequência de verificar-e-modificar ininterrupta para outras threads ou processos. O contador é um recurso compartilhado; acesso não atômico permite que atacantes contornem o limite. AES, RBAC e mTLS tratam confidencialidade, autorização e autenticação de transporte — não corrigem falhas de concorrência.

Questão 9: Alternativa A — Reverificar a autorização no momento da criação do recurso e exigir que a decisão de acesso esteja vinculada a um token assinado de uso único e curta duração obtido dentro da mesma transação

O gap TOCTOU está entre a checagem IAM e a criação da VM. Re-validar a autorização atomicamente no momento do uso — normalmente emitindo uma capability/token de curta duração dentro da transação verificada e resgatando-o na criação — elimina o gap. Rotação de chaves, VPC logs e IAM público não corrigem o drift concorrente de autorização e ainda podem enfraquecer a postura de segurança.

Questão 10: Alternativa A — Copiar o tamanho para uma variável local do kernel, validá-lo uma vez e passar essa cópia local validada a copy_from_user(), para que o valor não mude entre a checagem e o uso

Capturar uma cópia local do tamanho e validá-la antes de passá-la a copy_from_user() transforma o verificar-e-usar, antes não atômico, em uma operação atômica sobre memória controlada pelo kernel, eliminando o TOCTOU. Desabilitar preempção quebra garantias de tempo real. Criptografar userspace não impede que outra thread modifique a variável de tamanho, e mover a lógica para userspace expande a superfície de ataque.